ATHELEOS ARENA ENGINEERING | Version 1.0.2+4 | Production Architecture

# System Architecture & Tech Stack Specification

A Comprehensive Technical Reference for Atheleos: Sports-First Social Platform

| PRIMARY CLIENT | BACKEND RUNTIME | DATABASE | OBJECT STORAGE |
|---|---|---|---|
| Flutter (iOS & Android) | Firebase Functions & Cloudflare | Google Cloud Firestore | Cloudflare R2 Bucket |

## 1 Frontend Architecture & Client Stack

The client-facing platform is built with a high-performance, cross-platform **Flutter** architecture designed for media-rich feeds, fluid short-form video streaming, and low-latency interaction across athletes, coaches, scouts, and sports brands.

### Core Framework & Language

**Flutter SDK (Dart 3.x)** with compile-time null safety, multi-flavor build systems, and native C++ engine bindings for 60/120fps hardware-accelerated UI rendering on Android and iOS.

### State Management & DI

**Flutter Riverpod** ( `flutter_riverpod` , `riverpod_annotation` , `riverpod_generator` ). Provides compile-safe reactive state, dependency injection, and automatic provider disposal.

### Navigation & Deep Linking

**GoRouter** ( `go_router 14.x` ). Declarative route orchestration with sub-route hierarchies, parameterized URLs, deep linking, and real-time auth-state redirect guards.

### Media & Video Feed Engine

**Video Player & Visibility Detector** ( `video_player` , `visibility_detector` ). TikTok/Reels vertical feed autoplay/pause, viewport tracking, aspect-ratio correction, and memory-safe image cache.

| Module / Domain | Packages & Core Files | Key Technical Responsibility |
|---|---|---|
| **Feature Architecture** | `lib/features/` auth feed clips chat events profile upload | Clean architecture dividing application layers into Data, Domain, and Presentation (Controllers, StateNotifier, and Widgets). |
| **Design System & UI** | `google_fonts` , `shimmer` , `flutter_spinkit` , `assets/fonts/AthIcons.ttf` | Custom athletic dark theme, typography, skeleton loaders for image/post feeds, and bespoke SVG icon glyph font. |
| **Local Storage & Caching** | `shared_preferences` , `path_provider` , `core/services/cache_manager.dart` | Global memory cache cap (60MB/150 images), local key-value preferences, offline mutation queue, and LRU asset disk caching. |
| **Notifications & System** | `firebase_messaging` , `flutter_local_notifications` , `connectivity_plus` | FCM push token sync, timezone-scheduled local reminders, real-time connectivity listening, and native share intents. |

## 2 Backend & Serverless Compute

ATHELEOS uses a hybrid serverless model: **Firebase Cloud Functions** handle CPU-heavy operations (video transcoding, integrity triggers, cleanup crons) while **Cloudflare Workers** act as a global edge upload gateway with zero egress fees.

### Firebase Cloud Functions (Node.js 20)

Written in TypeScript ( `functions/src/index.ts` ). High-memory containers (2GB RAM) executing automated triggers, media jobs, and administration endpoints.

### Cloudflare Edge Worker

Deployed at `atheleos-media-worker` on Cloudflare's global CDN edge. Handles token validation, streaming uploads, deduplication, and rate limiting.

| Function / Service | Trigger / Protocol | Description & Operational Implementation |
|---|---|---|
| `processVideoJob` | HTTPS (Authenticated) | **FFmpeg Transcoding Pipeline:** Scales input video to 1080p/720p H.264/AAC, eliminates anamorphic distortion via `setsar=1`, generates poster thumbnails, parses aspect ratios, and executes 3x retries before dispatching to Dead Letter Queue ( `dead_letter_jobs` ). |
| `onPostWrite` `onStoryWrite` `onEventWrite` `onProfileWrite` | Firestore Document Triggers | **Automated Reference Counting:** Detects added or removed media IDs on documents, writes to `media_references` , and atomically increments or decrements `media.referenceCount` in Firestore transactions. |
| `dailyMediaCleanup` | Pub/Sub Cron (Every 24h) | **Orphan Garbage Sweep:** Queries media records with `referenceCount == 0` older than 24 hours, issues deletion requests to Cloudflare R2, removes Firestore documents, and repairs discrepancy counts. |
| `sendEmailOtp` `onEmailOtpWrite` | HTTPS & Firestore Trigger | **OTP Delivery Service:** Dispatches branded HTML verification codes via Hostinger SMTP with rate limiting and 10-minute expiry validation. |
| `cleanupIncompleteAccount` | HTTPS (Admin) | **Orphan Sanitization:** Purges abandoned Firebase Auth user records and incomplete profile drafts if phone/email verification steps were aborted. |

**Edge Media Gateway (Cloudflare Worker Pipeline)**

Mobile devices upload large media files directly to the Cloudflare Worker ( `/upload` ). The worker verifies the Firebase ID token at the edge, checks SHA-256 hashes to prevent redundant uploads, stages files into temporary R2 paths, and notifies `processVideoJob` for asynchronous background transcoding.

## 3 Database & Storage Layer

The persistence tier couples **Google Cloud Firestore** for real-time document synchronization and querying with **Cloudflare R2** for S3-compatible, zero-egress object storage.

### Google Cloud Firestore

Document-based NoSQL database with client-side offline persistence (100MB LRU cache). Governed by granular security rules ( `firestore.rules` ) and composite indexes ( `firestore.indexes.json` ).

### Cloudflare R2 Storage Bucket

Bucket: `atheleos-media` . Stores raw ingested video masters, transcode renditions, high-res photos, WebP thumbnails, and story assets with CDN caching.

| Firestore Collection | Key Document Fields | Business & Platform Role |
| --- | --- | --- |
| `users` | `uid` , `role` , `username` , `sports` , `verified` , `stats` | Multi-role profile models (Athlete, Coach, Brand, Federation) with verification status, bio, and follower metrics. |
| `posts` & `stories` | `media[]` , `caption` , `likesCount` , `commentsCount` , `expiresAt` | Feed items and 24-hour expiring stories containing video URLs, aspect ratios, thumbnails, and hashtag indexes. |
| `events` | `title` , `sport` , `dates` , `location` , `participants/` | Sports tournaments, trial dates, and matches with participant RSVP subcollections. |
| `chat_rooms` & `messages` | `participants[]` , `lastMessage` , `mediaUrl` , `readBy[]` | Direct and group messaging rooms with end-to-end receipt acknowledgments and media attachments. |
| `media` & `media_references` | `publicId` , `urls{}` , `referenceCount` , `sourceType` , `ownerId` | Media catalog maintaining video probes, encoding variants, and strict reference counts for automatic GC. |
| `processing_jobs` & `dead_letter_jobs` | `jobId` , `status` , `retryCount` , `videoMetadata{}` , `summary{}` | Asynchronous transcoding queues with progress milestones, durations, and error triage archives. |

## 4 Authentication, Email & Security

Security in ATHELEOS combines OAuth standards, short-lived one-time passwords, cryptographic token checks, and server-side authorization boundaries.

### Firebase Authentication & OAuth

Identity management supporting Email & Password authentication and Google Sign-In ( `google_sign_in` ). Issues signed RS256 JWT tokens with custom user claims.

### Hostinger SMTP Email Gateway

Dedicated TLS email relay ( `smtp.hostinger.com:465` ) configured with DKIM/SPF from `contact@atheleos.in` for OTPs and notifications.

### Edge JWT Verification

Cloudflare Worker cryptographically validates Firebase ID tokens directly against Google's public x509 certificates before admitting requests.

### Zero-Trust Firestore Security

`firestore.rules` strictly prevents client edits to verified badges, reference counters, report states, or impersonating other athletes.

| Workflow / Component | Implementation Detail | Security Guarantees |
| --- | --- | --- |
| **Email OTP Verification** | 6-digit cryptographically generated OTP stored in `email_otps/{email}` with timestamp. Dispatched via Nodemailer. | 10-minute expiry window, single-use invalidation upon verification, rate-limited resend cooldowns. |
| **Server Password Reset** | HTTPS endpoint `resetPasswordWithOtp` requires valid OTP matching the user before calling `admin.auth().updateUser()` . | Enforces strict password complexity (min 8 chars, uppercase, lowercase, numbers, special characters) entirely on server. |
| **Admin & Moderation Separation** | Role-based custom claims ( `admin` , `moderator` ) checked by Firestore rules and Cloud Functions. | Clients cannot approve athlete verification applications, delete foreign media, or modify sports taxonomy. |

```
// Example: Zero-Trust Verification Rule from firestore.rules
match /users/{userId} {
  allow read: if true;
  allow update: if request.auth.uid == userId
    && !request.resource.data.diff(resource.data).affectedKeys()
        .hasAny(['verified', 'verificationBadge', 'verificationLevel']);
}
```